



1 Introduction à l'algorithmique

1.1 Qu'est-ce qu'un algorithme ?

Un **algorithme** est une **suite finie d'instructions** permettant de résoudre un problème ou d'accomplir une tâche.

Exemple du quotidien

Pour faire cuire des pâtes, vous suivez une série d'étapes : faire bouillir l'eau, ajouter les pâtes, attendre 10 minutes, etc. C'est un **algorithme du quotidien**.

1.2 Pourquoi apprendre à concevoir des algorithmes ?

- Pour **décomposer** un problème en étapes logiques.
- Pour pouvoir ensuite **traduire** cette solution dans un langage de programmation.
- Parce qu'un **programme bien écrit repose toujours sur un algorithme bien pensé**.

2 Construction d'un algorithme

Une démarche classique et générique pour construire un algorithme :

1. **Comprendre le problème**
 - Lire et reformuler l'énoncé.
 - Identifier les **données d'entrée** et les **résultats attendus**.
2. **Imaginer une solution pas à pas**
 - Utiliser un raisonnement logique.
 - Réfléchir à la **séquence des opérations**.
3. **Écrire l'algorithme**
 - En langage naturel ou pseudo-code.
 - Utiliser des structures : affectation, condition, boucle, etc.
4. **Vérifier sa validité**
 - Le tester à la main avec des données simples.
 - S'assurer qu'il est **fini, correct et efficace**.

3 Représentation graphique : Logigramme

3.1 Qu'est-ce qu'un logigramme ?

Un **logigramme** (aussi appelé *organigramme* ou *algorithme*) est une **représentation visuelle d'un algorithme** sous forme de blocs reliés par des flèches.

b. Pourquoi utiliser un logigramme ?

- Pour **visualiser la logique** d'un programme ou d'un algorithme.
- Pour **communiquer facilement** une solution à d'autres personnes.
- Pour faciliter la **traduction vers du code**.

4 Blocs standards d'un logigramme

Bloc	Signification
Rectangle	Instruction / Action (ex : calcul, affichage)
Losange	Test / Condition (oui/non)
Ovale	Début / Fin
Flèche	Flux logique de l'exécution

